

claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a085344cafb3ef19f.jsonl`  
- SHA-256: `c00c4ee708c6ec3363dbbbee39eaff7d8867cd16ec47a759e66d9bd929425704`  
- Source modified: `2026-05-31T04:18:22+00:00`  
- Imported at: `2026-07-05T16:48:20+00:00`  
- project: `wf_076beba8-59c`  
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`
```

Transcript

1. user (2026-05-31T04:17:29.379Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):

1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:

- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:

```
**URL:** https://insiderllm.com/guides/vision-models-locally/  
**Title:** Best Vision Models You Can Run Locally: Every Model, Every GPU Tier (InsiderLLM)  
**Found via:** Open VLMs for document/table extraction (broad/primary) search
```

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
 - be a concrete, checkable statement (not vague generalities)
 - include a direct quote from the source as support
 - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:17:32.618Z)

I'll fetch the source page and extract key claims relevant to the research question.

3. user (2026-05-31T04:17:56.794Z)

Vision-Language Models for Local/Self-Hosted Document Processing

Publication Date: February 6, 2026 (Updated May 11, 2026)

Source Type: Independent technical blog (InsiderLLM)

Recommended VLM Models

Top Picks by VRAM Tier

24GB+ GPUs (State-of-the-Art):

- **Qwen 3.6-27B dense** (~17 GB VRAM, Apache 2.0): "the new state-of-the-art local vision model on 24GB+ GPUs"
- **Qwen 3.6-35B-A3B MoE** (35B total, 3B active, ~22 GB, Apache 2.0)
- **Gemma 4 31B dense** (~20 GB): "Highest vision benchmarks at this tier (MMMU Pro 76.9)"

```

**16-22 GB GPUs:**
- **Qwen 3.6-35B-A3B MoE** via `--cpu-moe` expert offload
- **Gemma 4 26B-A4B** (~18 GB, Apache 2.0): "Fast multimodal MoE, 3.8B active per token"

**8GB GPUs:**
- **Qwen2.5-VL 7B (Q4)**: "Proven 8GB fallback"
- **Qwen3-VL 8B**: "Best for 8GB GPUs"

**4GB GPUs:**
- **Gemma 3 4B (int4)**: "2.6 GB, real vision at minimum VRAM"
- **Smo1VLM2 2.2B**: "~2 GB, edge-grade but functional"

**Document/OCR Specialist (Any Hardware):**
- **PaddleOCR-VL 0.9B** (CPU-capable): "92.6% doc accuracy," runs on CPU

```

License Statements

Model	License
Qwen 3.6-27B dense	Apache 2.0
Qwen 3.6-35B-A3B MoE	Apache 2.0
Gemma 4 (all variants)	Apache 2.0
Qwen3-VL	(not explicitly stated in article)
Qwen2.5-VL	(not explicitly stated in article)

VRAM Requirements & Quantization Fit

****Full VRAM Table (excerpt):****

Model	Params	Q4 VRAM	Q8 VRAM	FP16 VRAM
Gemma 3 4B	4B	~2.6 GB	~5 GB	~8 GB
Qwen2.5-VL 7B	7B	~6 GB	~8 GB	~14 GB
Qwen3-VL 8B	8B	~6 GB	~9 GB	~16 GB
Phi-4-reasoning-vision	15B	~10 GB	~16 GB	~30 GB
Qwen3-VL 32B	32B	~21 GB	~34 GB	~64 GB

****Key note:**** "Vision models use more VRAM than text-only models of the same size because the vision encoder adds overhead"

Python Integration Paths

****Ollama (easiest):****

```

` `` bash
ollama run qwen3-vl:8b
ollama run gemma4:26b
ollama run qwen2.5vl:7b
` ``

```

****Python SDK (Ollama):****
 Shown with image handling via `images`: ['screenshot.png']` parameter

****llama.cpp (for Qwen 3.6 & Phi-4):****
 "Qwen 3.6 needs llama.cpp or LM Studio (Ollama support pending)"

****LM Studio:****
 Supports drag-drop image input; "supports Gemma 3, Qwen2.5-VL, LLaVA"

****No support mentioned for:**** vLLM (though PaddleOCR-VL mentioned as supporting vLLM backend separately)

Document/Table/OCR Reliability & Accuracy

Model	Metric	Score
PaddleOCR-VL 0.9B	OmniDocBench	92.6
PaddleOCR-VL 0.9B	Formula CDM	91.4
Qwen2.5-VL 7B	DocVQA	95.7
Qwen3-VL 8B	DocVQA	95+
Gemma 3 27B (QAT)	DocVQA	86.6

****Key claim:**** "Qwen2.5-VL 7B held the crown for months...Its 95.7 DocVQA score was the number that made people take local vision models seriously"

****PaddleOCR distinction:**** "When you need exact layout preservation ? tables, formulas, column structures...Qwen models are better when...answer questions about its content"

****Comparison to cloud:**** "Local matches cloud on document OCR and text extraction"

Notable Limitations Stated

- Qwen 3.6 GGUFs: "as of May 2026, Qwen 3.6 GGUFs still don't load in Ollama"
- Phi-4-reasoning-vision: "Not in Ollama's official library as of March 2026"
- LLaVA-OneVision: "not even in Ollama's official library"
- Vision generation: "They understand images. They don't generate them"

4. assistant (2026-05-31T04:18:02.151Z)

The fetch succeeded. Let me assess this source and extract the relevant claims.

This is an independent technical blog (InsiderLLM), published February 6, 2026, updated May 11, 2026. It provides specific, falsifiable claims about local VLMS with license, VRAM, and benchmark details relevant to muneem's bank-statement parsing question. Note one caution: some claims reference models ("Qwen 3.6", "Gemma 4") that may be speculative/forward-dated relative to the assistant's January 2026 knowledge cutoff ? I'll flag the source quality as blog accordingly and select the most concrete, verifiable claims.

5. user (2026-05-31T04:18:10.819Z)

Structured output provided successfully

6. assistant (2026-05-31T04:18:22.882Z)

I've completed the analysis of the InsiderLLM source.

Summary of findings:

Source quality: blog (independent technical blog, InsiderLLM). Published 2026-02-06, updated 2026-05-11. Not primary research or institutional ? it's a benchmark-aggregating technical blog. Notable caveat: several headline picks ("Qwen 3.6", "Gemma 4") are forward-dated and could not be independently verified against my knowledge, so I deliberately extracted only the most concrete, checkable claims (Qwen2.5-VL 7B, PaddleOCR-VL) and rated the source as blog rather than primary/secondary.

Claims extracted (most relevant to muneem's bank-statement parsing):

1. Central ? Qwen2.5-VL 7B is an 8GB-tier document VLM (~6 GB Q4 / ~8 GB Q8 / ~14 GB FP16) with

a 95.7 DocVQA score. Directly addresses the consumer-GPU VRAM tier and a candidate VLM the research question asks about.

2. Central ? PaddleOCR-VL 0.9B is the layout/table-structure specialist (92.6 OmniDocBench, CPU-capable, vLLM backend). Directly relevant to the "exact layout preservation ? tables, columns" need for semi-ruled bank tables.

3. Supporting ? The article's task split: layout-preserving OCR specialists beat general VLMs for structured tables, while VLMs win on content Q&A. Informs the "VLM vs OCR+table-structure pipeline" question.

4. Supporting ? Claim that local now matches cloud on document OCR/text extraction ? bears on muneem's local-first-vs-cloud-fallback framing.

5. Supporting ? Vision encoder overhead means VLMs need more VRAM than equal-size text models ? affects VRAM-tier budgeting.

License caveat for the parent agent: the blog only explicitly stated licenses for the unverifiable forward-dated models (Qwen 3.6 / Gemma 4 = Apache 2.0). It did NOT state weights licenses for Qwen2.5-VL or PaddleOCR-VL, the two models I extracted ? so those license claims must be verified at the source (HuggingFace model cards), not taken from this article. This source is useful for VRAM/benchmark framing but is not a reliable license authority.